

ATIVIDADE: OTIMIZAÇÃO DE QUERY

ALUNO: Jean Gustavo Vieira Guedes

TURMA: Engenharia da Computação - 6º Semestre (META)

PROFESSOR: Felipe Souza Damasceno

QUERY 1: SELECT * sem filtro (Caminhão de Lixo)

Problema original: Retorna colunas desnecessárias e sobrecarrega a rede (Full Table Scan com I/O excessivo).

Solução: Especificar estritamente as colunas necessárias e limitar o número de linhas retornadas.

```
SELECT id, nome, email, data_cadastro
```

```
FROM clientes
```

```
ORDER BY data_cadastro DESC
```

```
LIMIT 50;
```

QUERY 2: Violação do Leftmost Prefix no índice composto (estado, cidade)

Problema original: O índice não é acionado se a coluna primária (estado) não for informada.

Solução A: Incluir o estado na cláusula WHERE (se conhecido pela aplicação).

```
SELECT id_cliente, rua, bairro
```

```
FROM enderecos
```

```
WHERE estado = 'SP'
```

```
AND cidade = 'Campinas';
```

Solução B: Criar índice específico para a cidade, caso a busca isolada seja frequente.

```
CREATE INDEX idx_enderecos_cidade ON enderecos(cidade);
```

QUERY 3: Junção Tardia com Nested Loop pesado e funções em datas

Problema original: Aplicar função YEAR() no WHERE inviabiliza o uso de índices (Non-SARGable predicate).

Solução: Utilizar range de datas contínuo para permitir a indexação (Index Seek).

```
SELECT p.id, c.nome, p.valor_total  
FROM pedidos p  
INNER JOIN clientes c ON p.id_cliente = c.id  
WHERE p.data_pedido >= '2026-01-01'  
AND p.data_pedido < '2027-01-01';
```

QUERY 4: Desperdício de Covering Index com SELECT *

Problema original: Obriga o SGBD a ler a tabela física (Data Pages) para buscar as colunas ausentes no índice.

Solução: Selecionar apenas as colunas que compõem o índice, garantindo um Index Only Scan.

```
SELECT estado, cidade  
FROM userinfo  
WHERE estado = 'SP';
```

QUERY 5: A "Query Monstro" (Subconsulta correlacionada e DISTINCT desnecessário)

Problema original: A subquery no SELECT executa uma verificação linha por linha, e o DISTINCT gera custo extra de ordenação.

Solução: Resolver utilizando agrupamento (GROUP BY) e agregação (MAX) diretamente no JOIN.

```
SELECT
    c.id,
    c.nome,
    SUM(p.valor) AS total_gasto,
    MAX(p.data_pedido) AS data_ultimo_pedido
FROM clientes c
INNER JOIN pedidos p ON c.id = p.id_cliente
WHERE c.data_cadastro >= '2023-01-01'
    AND c.status = 'ativo'
GROUP BY c.id, c.nome
ORDER BY total_gasto DESC;
```

QUERY 6: Produto Cartesiano Velado (Sintaxe antiga com vírgula)

Problema original: A omissão da cláusula de junção cruza todos os registros (CROSS JOIN).

Solução: Utilizar a sintaxe explícita de INNER JOIN.

```
SELECT p.nome, c.nome_categoria
FROM produtos p
INNER JOIN categorias c ON p.id_categoria = c.id;
```

QUERY 7: Destruição do uso de índice em Árvore B+ por aplicação de funções (MONTH/YEAR)

Problema original: Assim como na Query 3, envelopar a coluna em funções força um Table Scan.

Solução: Escrever as condições em formato SARGable, delimitando o intervalo de tempo.

```
SELECT id, status, total
FROM faturas
WHERE data_criacao >= '2026-03-01'
AND data_criacao < '2026-04-01';
```

QUERY 8: Type Mismatch (Conversão implícita)

Problema original: Comparar um VARCHAR indexado com um INT obriga o banco a realizar um cast dinâmico, anulando o índice.

Solução: Envolver o valor buscado em aspas simples.

```
SELECT id, valor, status
FROM transacoes
WHERE codigo_transacao = '9845720194';
```

QUERY 9: Gargalo de I/O por Subconsultas aninhadas com IN

Problema original: Múltiplos INs aninhados costumam gerar planos de execução ruins e Full Table Scans.

Solução: Substituir por EXISTS. (Nota: O uso de EXISTS é superior ao INNER JOIN com DISTINCT, pois o SGBD para a busca assim que encontra a primeira correspondência, otimizando o processamento sem gerar duplicidades).

```
SELECT c.id, c.nome, c.email
FROM clientes c
WHERE EXISTS (
    SELECT 1
    FROM pedidos p
    INNER JOIN produtos pr ON p.id_produto = pr.id
```

```
WHERE p.id_cliente = c.id  
AND pr.categoria = 'Eletronicos'  
);
```

QUERY 10: LEFT JOIN limitando o otimizador no Nested Loop

Problema original: Aplicar uma condição na tabela da direita (pedidos.status = 'faturado') anula o LEFT JOIN, transformando-o logicamente num INNER JOIN, mas confundindo o otimizador.

Solução: Declarar explicitamente como INNER JOIN.

```
SELECT c.id, c.nome, p.id AS pedido_id, p.valor_total  
FROM clientes c  
INNER JOIN pedidos p ON c.id = p.id_cliente  
WHERE p.status = 'faturado'  
AND p.valor_total > 500;
```